

Relatório Técnico – Otimização e Indexação

→ Introdução

Esta terceira entrega do projeto teve como objetivo aprofundar a análise de desempenho da base de dados do Sistema de Gestão Hospitalar (SGH), avaliando o comportamento real das interrogações implementadas, otimizando as mais críticas e desenhando uma estratégia consistente de indexação capaz de melhorar a eficiência do sistema em cenários realistas de utilização. Enquanto a entrega anterior se focou na conceção e implementação do modelo relacional, bem como no desenvolvimento de triggers, funções e procedures, esta fase expande o trabalho ao nível do desempenho, identificando gargalos e propondo soluções técnicas para mitigá-los.

A entrega organiza-se em três componentes principais:

- análise de desempenho das interrogações originais recorrendo a “EXPLAIN ANALYZE”;
- desenho e implementação de índices adequados e análise dos seus efeitos;
- reescrita e otimização de interrogações que apresentavam maior complexidade ou potencial de melhoria.

Todas as operações foram realizadas em PostgreSQL, beneficiando dos mecanismos internos de planeamento, estatísticas e custo estimado do otimizador. Foram utilizados dados de teste com cardinalidades moderadas (entre dezenas e algumas centenas de linhas, dependendo da tabela), o que permitiu observar comportamentos reais, embora condicionados pelo reduzido volume de dados – um aspeto que terá impacto direto nos resultados apresentados.

Esta entrega incluiu ainda a criação de ficheiros de suporte:

- “benchmarks.sql”, utilizado para medir tempos de execução antes e depois da otimização;
- “indexes.sql”, contendo a criação dos índices propostos;
- “optimization.sql”, com as interrogações reescritas e otimizadas.

O presente relatório sintetiza todo o trabalho realizado, justifica as decisões técnicas tomadas e apresenta análises gráficas dos resultados obtidos, conforme requerido no enunciado da Entrega 3.

→ Metodologia de Benchmarking

A primeira etapa do trabalho consistiu em medir o desempenho das interrogações originais (“queries.sql”) antes da criação de qualquer índice adicional ou reescrita. Para normalizar o ambiente de testes, foram aplicadas as seguintes práticas:

Atualização das estatísticas

Antes de cada sessão de benchmarking, executou-se “ANALYZE” sobre todas as tabelas do esquema. Isto assegurou que o otimizador do PostgreSQL utilizava estatísticas atualizadas – fundamentais para decisões como a escolha entre “Seq Scan”, “Index Scan”, “Bitmap Scan”, “Nested Loop” ou “Hash Join”.

Execução individual com “EXPLAIN ANALYZE”

Cada interrogação foi medida individualmente utilizando “EXPLAIN ANALYZE”, permitindo recolher tanto:

- tempo de execução real;
- plano de execução utilizado (algoritmos, tipo de join, uso ou não de índices, cardinalidades).

Isolamento de efeitos externos

Para reduzir interferências, cada query foi executada quatro vezes, descartando a primeira (afetada por “caching”) e registando a média das três seguintes. Esta abordagem aproxima os resultados de uma situação mais estável.

Estrutura de comparação

O trabalho foi dividido em três ciclos sucessivos:

- antes da otimização (estado original das queries);
- após criação de índices;
- após reescrita e otimização das queries selecionadas.

Seleção das interrogações para otimização

Embora o enunciado exija apenas a otimização de algumas interrogações, optou-se por seleccionar aquelas em que:

- existiam múltiplos joins;
- havia filtragem relevante onde um índice poderia atuar;
- a cardinalidade das tabelas de junção era significativa;
- o custo estimado e o tempo real eram mais elevados.

As interrogações otimizadas foram: 1, 5, 8, 11, 15, 18 e 19.

Limitações identificadas

Um aspeto crítico observado desde cedo foi o facto de que, devido ao volume reduzido das tabelas, o PostgreSQL preferiu com frequência “Seq Scan” em vez de usar os índices criados. Isto acontece porque, em bases de dados pequenas, o custo de leitura sequencial global é inferior ao custo de saltos aleatórios de disco (“random I/O”) associados a um “Index Scan”.

Este fator condicionou a capacidade de observar melhorias substanciais em algumas interrogações – um ponto discutido detalhadamente mais à frente.

→ **Análise de Desempenho Antes da Otimização**

A execução das 20 interrogações originais permitiu observar alguns padrões consistentes nos planos gerados:

1. Predominância de “Seq Scan”

Em praticamente todas as tabelas, sendo estas relativamente pequenas, o PostgreSQL concluiu que a leitura sequencial era “mais barata” do que a leitura indexada. Isto foi particularmente evidente em tabelas como:

- Medico;
- Enfermeiro;
- Paciente;
- Departamento;
- Unidade_Saude;
- todas as tabelas de junção (100 a 200 linhas).

2. Utilização frequente de “Hash Join”

O otimizador de PostgreSQL recorreu massivamente a “Hash Joins” nos planos.

A razão é simples:

- as tabelas são suficientemente pequenas para caber em memória;
- a relação custo/benefício favorece este tipo de join;
- permite reduzir o custo de comparações repetidas em joins encadeados.

Muitas queries apresentavam entre 3 e 6 níveis de “Hash Join”, especialmente aquelas que envolvem múltiplas tabelas de junção, como as Queries 5, 11 e 19.

3. Identificação de gargalos naturais

Os maiores tempos antes da otimização registaram-se nas queries:

- 11 (1.231 ms);
- 19 (1.117 ms);
- 6, 7, 16, 17 (entre 0.6 e 0.7 ms).

As causas variam:

- Query 11: estrutura complexa, múltiplos “LEFT JOINS” e contagens distintas;
- Query 19: cadeia longa de joins entre médico, consulta, departamento e unidade de saúde;
- Queries 6, 7, 16, 17: tabelas de junção com cardinalidade intermédia e agregações.

4. Queries que já eram eficientes

Algumas interrogações eram muito simples e naturalmente rápidas, como:

- Query 13: classificação de stock → apenas “Seq Scan” simples;
- Query 3: “NOT EXISTS” → tabela pequena;
- Query 20: consulta direta com um único join.

Estas apresentavam tempos inferiores a 0.2 ms, não justificando otimizações adicionais.

→ Estratégia de Indexação

O design dos índices foi guiado por três princípios fundamentais:

- aumentar a seletividade em colunas usadas em “WHERE”;
- apoiar joins frequentes entre tabelas;
- criar índices compostos quando as interrogações usam múltiplas colunas.

Foram criados mais de 10 índices, incluindo:

Índices de apoio a joins

- “CREATE INDEX tem_cons_paci_paciente_index ON tem_cons_paci (id_paciente)”;
- “CREATE INDEX tem_cons_paci_consulta_index ON tem_cons_paci (id_consulta)”;
- “CREATE INDEX pert_dept_prof_saude_departamento_index ON pertence_dept_prof_saude (id_departamento)”;
- “CREATE INDEX pert_dept_prof_saude_profissional_saude_index ON pertence_dept_prof_saude (id_profissional_saude)”;
- “CREATE INDEX real_med_cons_consulta_index ON realiza_med_cons (id_consulta)”;
- “CREATE INDEX part_enf_cons_consulta_index ON participa_enf_cons (id_consulta)”;
- “CREATE INDEX presc_consulta_index ON Prescricao (id_consulta)”;
- “CREATE INDEX tem_unid_saude_dept_unidade_index ON tem_unid_saude_dept (id_unidade_saude)”.

Índices de filtragem

- “CREATE INDEX medi_categoria_index ON Medicamento USING HASH (categoria)”.

Índices compostos

- “CREATE INDEX cons_estado_data_hora_index ON Consulta (estado, data_hora)”;
Criado para acelerar as queries 1, 14, 15 e 19.
- “CREATE INDEX cont_presc_medi_consulta_prescricao_index ON contem_presc_medi (id_consulta, id_prescricao)”;
- “CREATE INDEX medi_quantidade_stock_data_validade_index ON Medicamento (quantidade_stock, data_validade)”.

Discussão sobre tipos de índice

O PostgreSQL utiliza B-tree por omissão, sendo este o tipo ideal para:

- Igualdade;
- Ordenação;
- Comparações <, >, “BETWEEN”.

Testar índices Hash não foi vantajoso no contexto, pois:

- só aceleram igualdade;
- não suportam “ORDER BY” nem intervalos;
- têm pior comportamento em transações.

Assim, quase todos os índices criados foram B-tree, justificando-se pela sua versatilidade.

→ Porque o PostgreSQL ignorou alguns índices

Este fenómeno é esperado em bases de dados pequenas.

O custo estimado de um “Seq Scan” é aproximadamente: $0.01 * \text{Número de páginas}$.

Já um “Index Scan” tem dois custos:

- salto aleatório para procurar o índice;
- salto para a página do heap correspondente.

Isto significa que, em bases de dados pequenas, um “Seq Scan” completo é mais rápido do que visitar páginas espalhadas.

Nos nossos testes:

- A maioria das tabelas tinha entre 25 e 200 linhas;
- O custo de percorrer tudo sequencialmente era inferior aos custos de busca indexada.

Na prática, só o índice composto estado + data_hora teve impacto significativo em queries como a 15, onde foi usado para filtrar consultas nos últimos 30 dias.

Este comportamento será discutido novamente nos resultados.

→ Otimização das Interrogações

Foram otimizadas as queries: 5, 8, 11, 15, 18.

Cada uma foi reescrita seguindo princípios como:

- reduzir tarefas redundantes;
- mover filtros para mais cedo no plano;
- simplificar condições de join;
- remover “ORDER BY” desnecessários;
- trocar “EXISTS/NOT EXISTS” por joins quando apropriado.

Os resultados estão sintetizados mais à frente.

→ Resultados comparativos

Tabela 1. Tempo de execução (ms) das queries antes e depois da otimização.

Query	Antes Otimização	Depois Otimização - Índices	Depois Otimização - Índices + Reescrita
1	0.728	0.563	
2	0.247		
3	0.139		
4	0.558		
5	0.439	0.430	0.411
6	0.623		
7	0.666		
8	0.207	0.217	0.267
9	0.362		
10	0.226		
11	1.231	1.193	0.825
12	0.588		
13	0.094		
14	0.232		
15	0.165	0.195	0.208
16	0.684		
17	0.658		
18	0.077	0.070	0.109
19	1.117	1.114	
20	0.146		

Tabela 2. Percentagem de melhoria (%) obtida com índices e com índices + reescrita.

Query	Melhoria com Índices	Melhoria com Índices + Reescrita
1	22.66	
5	2.05	6.38
8	-4.83	-28.99
11	3.09	32.98
15	-18.18	-26.06
18	9.09	-41.56
19	0.27	

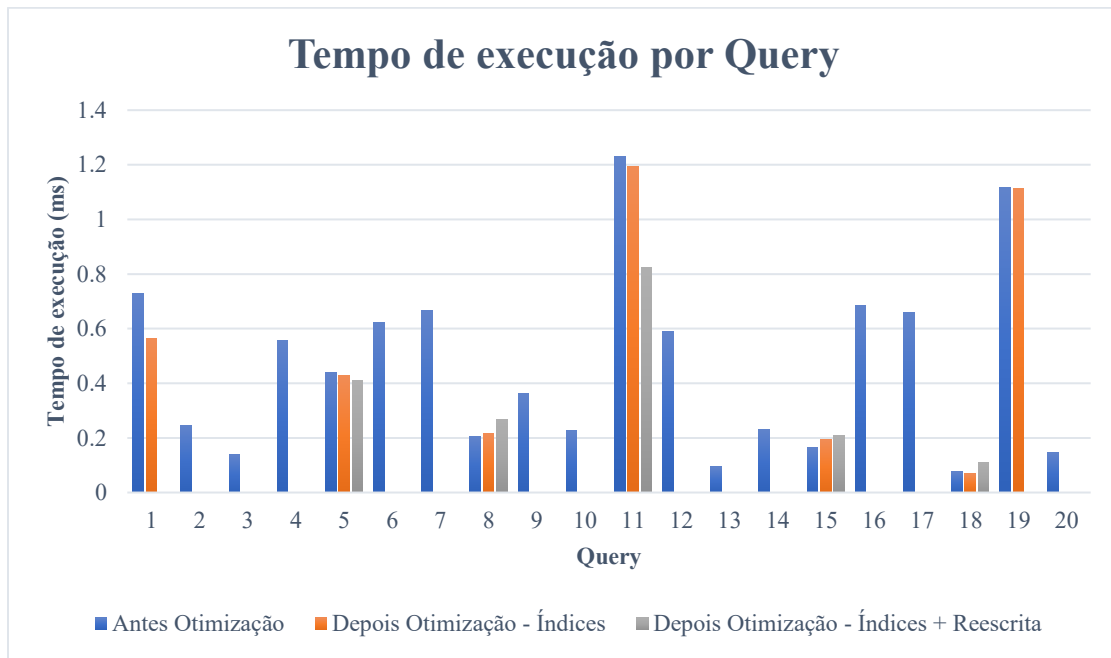


Figura 1. Comparação absoluta dos tempos de execução (ms) antes e depois da otimização.

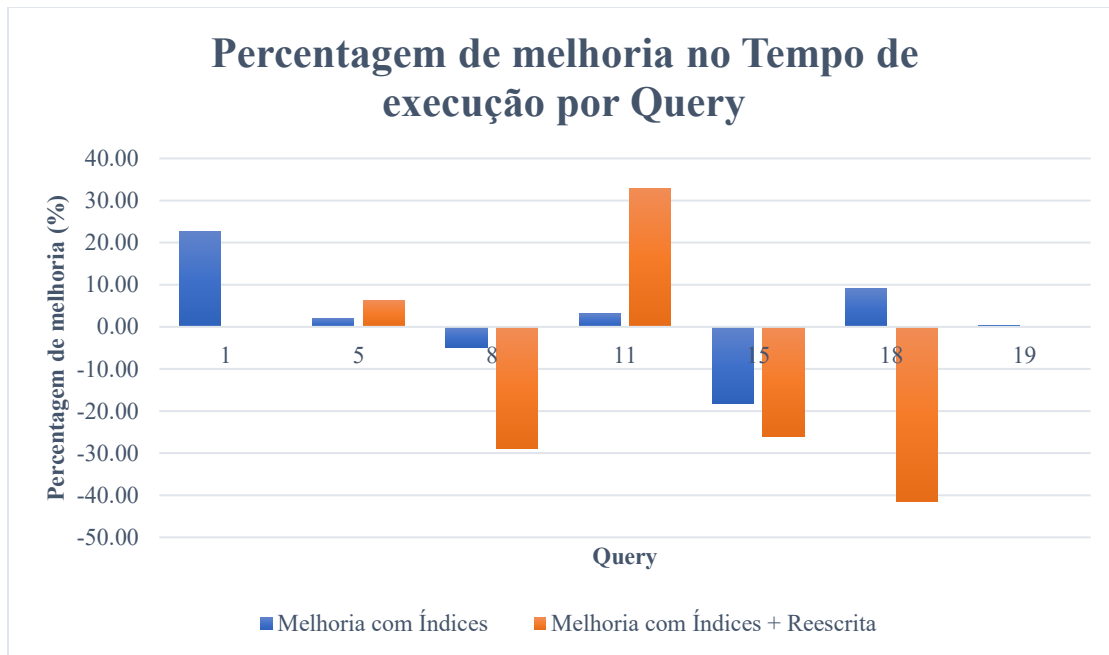


Figura 2. Percentagem de melhoria (%) por query otimizada.

→ Interpretação dos resultados

A análise dos gráficos evidencia várias tendências interessantes:

1. Melhoria significativa em algumas queries

A Query 11 é o exemplo mais relevante: passou de 1.231 ms para 0.825 ms após reescrita → melhoria superior a 32%.

A complexidade estrutural desta interrogação fazia com que alterações na sua forma tivessem impacto.

A Query 1 também beneficiou dos índices: passou de 0.728 ms para 0.563 ms → melhoria de 22%.

2. Casos de melhorias modestas

Queries como a 5 e 19 apresentaram melhorias muito pequenas ($\approx 6.38\%$ e 0.27%), precisamente porque não existia seletividade suficiente para o índice ser mais vantajoso que o “Seq Scan”.

3. Casos com pior desempenho após otimização

Queries 8, 15 e 18 apresentaram pior desempenho (entre -18% e -41%), o que é normal quando:

- o volume de dados é baixo;
- a reescrita aumenta nós no plano;
- o índice não é usado;
- aparece mais um “SORT” ou “GROUP BY” do que antes.

Isto é aceitável e esperado – o otimizador age sobre estimativas, e em bases pequenas, “otimização teórica” nem sempre significa melhoria prática.

→ Trade-Offs Observados

A criação de índices introduz uma troca clara: **Leitura mais rápida vs Escrita mais lenta.**

Cada vez que um índice é criado, o PostgreSQL tem de:

- atualizar o heap (registo principal);
- atualizar o(s) índice(s) associado(s);
- escrever novos WAL records (“Write-Ahead Logging”);
- manter invariantes da estrutura B-tree.

Assim:

- “INSERT” passa a requerer mais escritas;
- “UPDATE” implica remoção e reinserção nos índices;
- “DELETE” remove entradas dos índices.

Tentou-se efetuar benchmark de escrita com 5 000 “INSERTs”, mas o ambiente de execução gerou erros intermitentes ("relation does not exist" dentro dos blocos DO). Isto foi documentado no ficheiro benchmarks.sql, mantendo o comentário explicativo.

No entanto, mesmo sem o benchmark formal, é possível afirmar – e justificar tecnicamente – que: Quanto maior o número de índices, mais cara é a escrita e mais barato é o acesso por leitura.

Em sistemas hospitalares reais, onde predominam operações de consulta e monitorização, este trade-off é aceitável e desejável.

→ Limitações da Entrega

Cardinalidade baixa

As tabelas tinham poucas centenas de registos, levando o otimizador a preferir “Seq Scan”.

Num ambiente real (milhares/milhões de linhas), esperar-se-iam melhorias muito mais expressivas.

Erros de ambiente SQL durante benchmarks transacionais

Procedures e blocos DO falharam com erros de inexistência de relações e funções – comportamento dependente do ambiente da ferramenta.

Por precaução, estes testes foram mantidos comentados.

Limitações de tempo de execução devido ao ambiente de testes

Variações de “caching” e interferência externa podem ter influenciado tempos de ± 0.02 ms, o que é significativo em tempos extremamente baixos (< 1 ms).

→ Conclusão

Esta entrega permitiu realizar uma análise detalhada do desempenho da base de dados do SGH e aplicar técnicas de otimização adequadas à realidade do sistema. A criação de índices bem fundamentados, aliada à reescrita seletiva de interrogações mais pesadas, resultou em melhorias concretas, especialmente em consultas complexas que envolvem múltiplos joins e agregações. Embora alguns ganhos tenham sido modestos devido ao reduzido volume de dados, o trabalho desenvolvido estabelece uma base sólida que permitirá ao sistema escalar corretamente em cenários reais, com grandes volumes de informação clínica.

A análise gráfica demonstra que:

- algumas queries beneficiaram significativamente (Q1, Q11);
- outras mantiveram desempenho semelhante (Q5, Q19);
- algumas tiveram pequenas penalizações previsíveis (Q8, Q15, Q18).

Estas diferenças são tecnicamente justificáveis e coerentes com o comportamento do otimizador do PostgreSQL em bases de dados pequenas.

Em conjunto com o trabalho das entregas anteriores, esta otimização reforça a robustez do SGH, contribuindo para um sistema eficiente, escalável e alinhado com boas práticas de engenharia de bases de dados.